York University

Fall 2026 Course Project: Stage 2

AI News Analysis Agent

Nicole Van

219563162

EECS 3311 A - Software Design

Dr. Song Wang

November 8, 2026

1. Project Overview and Implementation Status

1.1 Application Overview

The AI News Analysis Agent is a Java based system that retrieves news articles from multiple sources, analyzes them with a locally hosted large language model (LLM), and provides users with consolidated cross source analysis of current events. Rather than simply presenting a feed of raw articles, the system can group related coverage, compare how different outlets report the same event, extract entities and claims, and produce summaries, timelines, and personalized briefings. The application exposes its functionality through both a Java Swing graphical user interface (GUI) and a text-based command line interface (CLI) so that the same core logic can be exercised through two different front ends.

1.2 Target Users

- **Citizens:** People who want to be informed about the current events in the world around them without needing to spend a lot of time manually checking/reading multiple news sources.
- **Journalists:** People who must quickly find and track new stories, verify information, and inform the public.
- **Industry Analysts:** People who must monitor specific topics, trends, and developments to provide insights and recommendations.

1.3 AI/LLM Model(s) and Architecture

The system will integrate the AI agent with a locally hosted big language model using the Java framework LangChain4j, with Ollama as the local LLM runtime. The system does not depend on any paid cloud LLM API. Instead, a locally hosted open-source model (Llama3) is used via Ollama's HTTP API on 'http://localhost:11434'.

Architecture:

Java Application → LangChain4j → Ollama → Llama3

LangChain4j interactions:

- Used for LLM communication
- Used for AI agent/tool integration
- Used for tool calling
- Used for structuring AI responses

1.4 Limitations and Incomplete Functionality

- **Local LLM Performance:**
  Since I use a locally hosted LLM(Llama3) which is downloaded onto my machine, processing can be slow sometimes. I could have fixed this by perhaps implementing a cloud-based LLM API instead, but as mentioned in my stage 1 report, I wanted to keep this news analysis agent free of cost without any worry of a quota. Thus, I decided not to go this route.
- **Volatile Memory:**
  The saved functionality works while the application is running. However, when the application is closed and is then run again, the previously saved data is cleared. I know this is due to me saving the data in a Java List within my repositories, so the data lives only in my machine's RAM. I could have fixed this by making the save function save the data into a physical file or an actual database server. Unfortunately though, I did not have enough time to implement this. (Note: The following applies for the personalized feed feature as well)

- **Bottleneck & Output Inaccuracy:**
  When fetching the news articles and scraping the content out, there tends to be more than 3000 characters on average. This greatly slows down the LLM's analysis speed. One way I could have fixed this, would have been to increase the timeout of the LLM client to avoid a timeout error. While this solution may work, the waiting time of the user would be ridiculously long. As such, I instead truncated the content to be 3000 characters maximum if the article contained more than that amount. There is still a bit of waiting, but I no longer get any timeout errors on my machine. One downside of this method though is that my analysis strategies depend on the entirety of the article. If the content gets truncated, the accuracy of the output is reduced as everything beyond the 3000 characters is trimmed off.
- **Unsupported Tool Calling:**
  I implemented the natural-language query functionality only to realize during testing that the model I chose (i.e., Ollama Llama3), does not support the tool calling feature that LangChain4j needs for my feature 15 (Natural-Language News Queries) to work. This could have been fixed by changing the LLM model. Unfortunately though, I did not have enough time to test which model would work in order to correct it. I have left the code for this feature within the project, so that an LLM model change could be made in the future.
- **API Key Quota (Free Tier Users):**
  My feature 12 (Topic Tracking) is required to run automatically in the background, where it has a timer that triggers the run every 2 minutes. The issue with this though is that it spams my NewsAPI quota (free tier = 100 requests/day) and triggers long LLM calls repeatedly. This is not ideal, so the timer is commented out in my code in NewsDashboard.java, and a temporary refresh button was added for the demo.
- **Daily Briefing Generation Inaccuracy:**
  When the user requests a daily briefing, AgentController.generateBriefing() will ask the NewsAnalysisAgent to gather and analyze the relevant current articles. The agent retrieves the available articles and uses the Clustering strategy to group the related articles into EventCluster objects. The articles associated with each cluster can then be processed by the Summarization strategy to generate a concise summary of each before combining them together to create the briefing.
  Unfortunately though, this summarization process is incredibly long (i.e., 5+ minutes) especially with each article containing thousands upon thousands of words. To combat this, I modified my feature 16 (Daily Briefing Generation) to skip the LLM summarization entirely, and to instead extract the first 1-2 sentences from each article using pure Java. With this method, the daily briefing generation takes only a second. The trade-off here however is that the briefing becomes less accurate/detailed as it does not actually analyze the entirety of each article. Another alternative fix I could have done would have been to switch my LLM model (Ollama Llama3) to a smaller one.
- **Clustering Bug:**
  In my feature 6 (Event Clustering), there is an issue where if the user chooses to 'detect duplicate stories/clusters' more than 6 times for 6+ different news fetches (NewsAPI and/or RSS), the 'CLUSTER 6' output will start to absorb all the newly generated articles whether related or not. No new cluster will be created. I have spent many hours trying to fix this issue, but have not been successful unfortunately.
- **Not Fully Realized CLI Implementation:**
  At this point, my CLI implementation is fairly bare-bones compared to my more polished GUI. There is currently only news fetching, summarization, and some simple error checks. If I had more time, I would

translate more of the features over. This should be reasonably simple as the GUI and CLI share the exact same facade as mentioned in my design patterns, so business logic is only written once.

1.5 Initial Implementation Plan

Week 1 (Oct. 20 - 26): Project foundation and basic analysis

- **Goal:** Establish the project structure and verify that the core components work independently
    - Setup the Java project using Maven (pom.xml) to integrate LangChain4j and Ollama
    - Install and configure Ollama, download llama3 Model, and test
    - Implement the domain models: Article, Topic, Claim, and Entity ("Model" in MVC design pattern)
    - Create the ArticleRepository and TopicRepository for storing and retrieving data
    - Define the generic AnalysisStrategy<T, R> interface and implement Summarization as the first concrete strategy (Strategy design pattern)

Week 2 (Oct. 27 - Nov. 2): News retrieval and agent orchestration

- **Goal:** Build the first complete workflow from retrieving an article to analyzing it
    - Implement the NewsSourceFactory and NewsSourceAdapter to fetch articles (Factory and Adapter design patterns)
    - Implement the RSSAdapter using Rome and NewsAPIAdapter
    - Build the NewsAnalysisAgent to coordinate retrieval and analysis
    - Implement the AgentController as a simplified entry point to the agent subsystem (Facade design pattern)
    - Implement a basic GUI/CLI using Java Swing GUI toolkit that lets users retrieve articles and request summaries

Week 3 (Nov. 3 - 9): Expanded analysis and integration

- **Goal:** Expand the agent's analytical capabilities and integrate more of the Stage 1 design
    - Implement additional AnalysisStrategy classes: Categorization, EntityExtraction, ClaimExtraction, and Comparison
    - Implement duplicate-story detection and event clustering
    - Add source linking and basic timeline generation
    - Add topic creation, topic tracking, and saved article functionality
    - Improve the Swing GUI and CLI to represent to newly implemented features
    - Update the README with setup instructions, and Ollama configuration

---

2. Feature to Implementation Mapping

Account for every Stage 1 feature and every new feature. Identify the classes and files that implement each feature, their responsibilities, relevant methods, design patterns, and demonstration video.

F01 — Topic Creation:

- **Status:** Implemented
- **Design Patterns:** MVC, Facade
- **Implemented Classes/Files/Responsibilities:**

    - NewsDashboard — provides the “Create Topic” and “Manage Topics” buttons; prompts the user for a topic name; displays the current topic list; triggers a first run topic creation dialog when no topics exist
    - AgentController — coordinates the topic creation request from the user interface
    - NewsAnalysisAgent — implements the business logic for creating a topic, avoiding duplicates by name, and deleting
    - TopicRepository — in-memory storage of Topic objects
    - Topic — model representing a news topic (id, name, keywords, tracked flag)
- **Important Methods:**
  createTopic(), getAllTopics(), deleteTopic(), save(), findAll(), findByName(), delete()
- **Video Link/Timestamp:**
    - **GUI Video:** **EECS3311_GUI.mp4**
        - **Timestamp:** 0:00:25 - 0:01:26

F02 — Article Retrieval:

- **Status:** Implemented
- **Design Patterns:** Factory Method, Adapter, Facade
- **Implemented Classes/Files/Responsibilities:**
    - NewsDashboard — provides the “Fetch News (NewsAPI)” and “Fetch News (BBC RSS)” buttons plus the search field text
    - AgentController — coordinates the article retrieval from external sources
    - NewsAnalysisAgent — orchestrates retrieval and saves results to the article repository
    - NewsSourceFactory — creates the appropriate source adapters based on the source type and query; routes known BBC categories to specific BBC feeds, and unknown queries to Google News RSS
    - NewsSourceAdapter — common interface for all the news source providers
    - NewsAPIAdapter — calls newsapi.org, parses JSON via Jackson, scrapes full text via Jsoup; truncates article to 3000 characters
    - RSSAdapter — parses RSS via Rome and scrapes full article text via Jsoup; falls back to description if scraping is blocked; truncates article to 3000 characters
    - ArticleRepository — stores and retrieves the Article objects
    - Article — model representing a news article (id, title, content, sourceUrl, publishedDate, category, saved)
- **Important Methods:**
  fetchNews(), retrieveArticles(), createAdapter(), pickFeedForQuery(), fetchArticles(), scrapeArticle(), save()
- **Video Link/Timestamp:**
    - **GUI Video:** **EECS3311_GUI.mp4**
        - **Timestamp:** 0:01:28 - 0:02:32
    - **CLI Video:** **EECS3311_CLI.mp4**
        - **Timestamp:** 0:00:49 - 0:01:30, 0:03:43 - 0:04:02

F03 — Article Categorization:

- **Status:** Implemented
- **Design Patterns:** Strategy, Facade
- **Implemented Classes/Files/Responsibilities:**
    - ArticleViewer — “Categorize” button; displays the resulting category
    - AgentController — provides the application-level interface for requesting AI analysis

    - NewsAnalysisAgent — coordinates the analysis process and selects the appropriate analysis strategy
    - AnalysisStrategy — a generic interface for each analysis strategy
    - Categorization — LLM backed categorization into a fixed set (Politics, Technology, Sports, Business, Health, Science, Entertainment, World)
    - LLMClient — provides access to the language model used for AI processing
    - Article — source of content
- **Important Methods:**
  categorize(), analyze(), generate()
- **Video Link/Timestamp:**
    - **GUI Video:** **EECS3311_GUI.mp4**
        - **Timestamp:** 0:03:57 - 0:04:25 (output at 0:04:18)

F04 — Article Summarization:

- **Status:** Implemented
- **Design Patterns:** Strategy, Facade
- **Implemented Classes/Files/Responsibilities:**
    - ArticleViewer — “Summarize” button; shows the 3 bullet AI summary
    - AgentController — provides the application-level interface for requesting article analysis
    - NewsAnalysisAgent — coordinates the analysis and selects the appropriate strategy
    - AnalysisStrategy — a generic interface for each analysis strategy
    - Summarization — condenses each article into a 3 bullet point summary
    - LLMClient — sends prompts to the language model and returns generated results
    - Article — source of content
- **Important Methods:**
  summarize(), analyze(), generate()
- **Video Link/Timestamp:**
    - **GUI Video:** **EECS3311_GUI.mp4**
        - **Timestamp:** 0:03:12 - 0:03:56
          (output at 0:03:48)
    - **CLI Video:** **EECS3311_CLI.mp4**
        - **Timestamp:** 0:01:42 - 0:03:42 (output at 0:03:34)

F05 — Duplicate-Story Detection:

- **Status:** Implemented
- **Design Patterns:** Strategy
- **Implemented Classes/Files/Responsibilities:**
    - NewsDashboard — “Detect Duplicate Stories / Clusters” button; renders multiple article clusters grouped by title and first few characters of each article’s content
    - AgentController — provides the application-level interface for requesting article analysis
    - NewsAnalysisAgent — coordinates the analysis required to identify similar articles
    - AnalysisStrategy — a generic interface for each analysis strategy
    - Clustering — implements pairwise Jaccard similarity over title + content word sets with stopword removal, host bonus, and union find grouping
    - ArticleRepository — provides the articles that are compared
    - Article — the articles being compared
- **Important Methods:**
  clusterStories(), clusterArticles(), analyze(), combinedSimilarity(), jaccard(), find(), getAllStoredArticles()

- **Video Link/Timestamp:**
    - **GUI Video: EECS3311_GUI.mp4**
        - **Timestamp:** 0:07:13 - 0:07:38

F06 — Event Clustering:

- **Status:** Implemented
- **Design Patterns:** Strategy
- **Implemented Classes/Files/Responsibilities:**
    - NewsDashboard — “Detect Duplicate Stories / Clusters” button; renders multiple article clusters grouped by title and first few characters of each article’s content
    - AgentController — provides the application-level interface for requesting article analysis
    - NewsAnalysisAgent — coordinates the event clustering analysis
    - AnalysisStrategy — a generic interface for each analysis strategy
    - Clustering — implements pairwise Jaccard similarity over title + content word sets with stopword removal, host bonus, and union find grouping
    - ArticleRepository — provides the articles that will be clustered
    - Article — the articles that are being grouped into clusters
- **Important Methods:**
  clusterStories(), clusterArticles(), analyze(), getAllStoredArticles()
- **Video Link/Timestamp:**
    - **I Video: EECS3311_GUI.mp4**
        - **Timestamp:** 0:07:13 - 0:07:38

F07 — Entity Extraction:

- **Status:** Implemented
- **Design Patterns:** Strategy, Facade
- **Implemented Classes/Files/Responsibilities:**
    - ArticleViewer — “Extract Entities” button; shows comma separated people/organizations/locations
    - AgentController — provides the interface for requesting AI analysis
    - NewsAnalysisAgent — coordinates entity extraction and selects the appropriate strategy
    - AnalysisStrategy — a generic interface for each analysis strategy
    - EntityExtraction — LLM backed extraction of named entities
    - LLMClient — provides access to the language model used for extraction
    - Entity — represents an extracted entity (person, organization, location, etc.)
    - Article — the article content where entities are being extracted from
- **Important Methods:**
  extractEntities(), analyze(), generate()
- **Video Link/Timestamp:**
    - **GUI Video: EECS3311_GUI.mp4**
        - **Timestamp:** 0:04:26 - 0:05:06 (output at 0:04:56)

F08 — Timeline Generation:

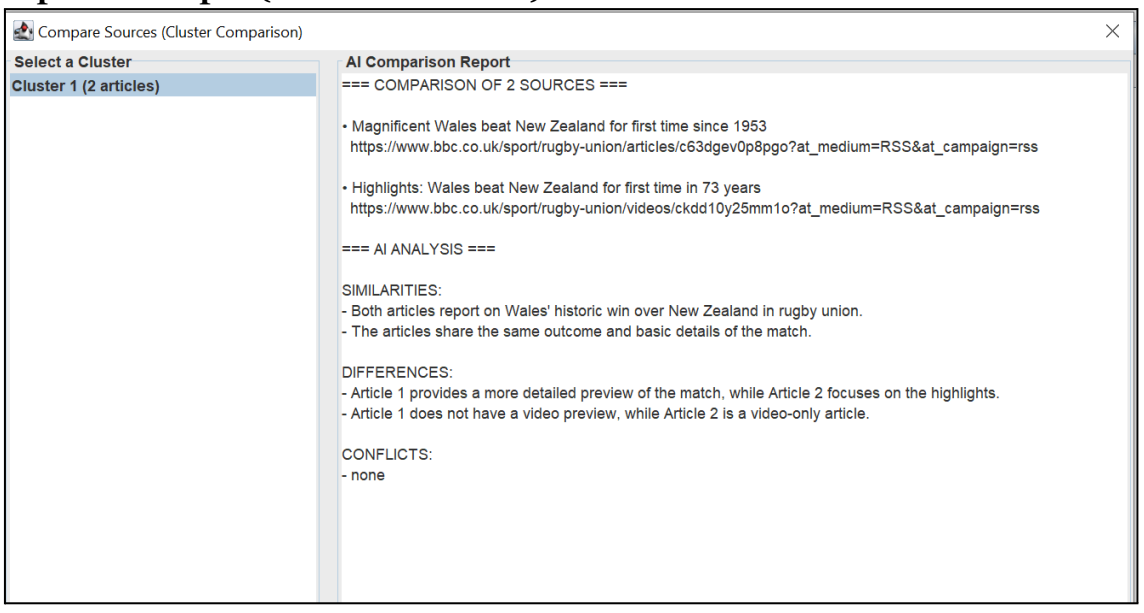
- **Status:** Implemented
- **Design Patterns:** MVC
- **Implemented Classes/Files/Responsibilities:**
    - NewsDashboard — “View Timeline” button; displays a scrollable dialog of events sorted chronologically

- AgentController — provides the application-level operation for requesting a timeline
- NewsAnalysisAgent — coordinates the analysis required to make the timeline
- Timeline — holds TimelineEvent objects; returns them sorted by date
- TimelineEvent — represents a single event (id, description, date)
- ArticleRepository — supplies the underlying articles

- **Important Methods:**
  generateTimeline(), getEvents(), addEvent(), getDate(), getAllStoredArticles()
- **Video Link/Timestamp:**
  - **GUI Video:** **EECS3311_GUI.mp4**
    - **Timestamp:** 0:12:01 - 0:12:22

F09 — Multiple Source Comparison:

- **Status:** Implemented
- **Design Patterns:** Strategy, Facade, MVC
- **Implemented Classes/Files/Responsibilities:**
  - NewsDashboard — "Compare Sources" button that opens ComparisonPanel
  - ComparisonPanel — dialog for selecting a cluster, invoking the comparison, and displaying a structured report with clickable source links
  - NewsAnalysisAgent — coordinates the comparison process
  - AnalysisStrategy — a generic interface for each analysis strategy
  - Comparison — LLM backed comparison producing fixed SIMILARITIES / DIFFERENCES / CONFLICTS structure
  - ClaimExtraction — used within the comparison pipeline for claim extraction
  - LLMClient — provides language model processing for comparison and claim analysis
  - SourceLinkManager — retrieves and opens source URLs
  - Article — the articles being compared
- **Important Methods:**
  compareArticles(), compare(), analyze(), getSourceLink(), openSourceLink()
- **Video Link/Timestamp:**
  - **GUI Video:** **EECS3311_GUI.mp4**
    - **Timestamp:** 0:07:43 - 0:11:54 (timeout error at 0:11:09 ⇒ screen recording while calling on LLM consumes resources! This feature will work when the screen is not being recorded.)
    - **Expected Output (No screen record):**



F10 — Claim Extraction:

- **Status:** Implemented
- **Design Patterns:** Strategy, Facade
- **Implemented Classes/Files/Responsibilities:**
  - ArticleViewer — “Extract Claims” button
  - NewsAnalysisAgent — coordinates the claim-extraction process
  - AnalysisStrategy — a generic interface for each analysis strategy
  - ClaimExtraction — LLM backed extraction of factual claims
  - LLMClient — provides language model processing for extracting the claims
  - Claim — represents an extracted claim along with its associated information
  - Article — the source of content
- **Important Methods:**
  extractClaims(), analyze(), generate()
- **Video Link/Timestamp:**
  - **GUI Video:** **EECS3311_GUI.mp4**
    - **Timestamp:** 0:05:07 - 0:06:07 (output at 0:06:00)

F11 — Source Linking:

- **Status:** Implemented
- **Design Patterns:** N/A
- **Implemented Classes/Files/Responsibilities:**
  - ArticleViewer — “View Source” button plus clickable URL label
  - ComparisonPanel — generates a clickable button per cluster article that opens the original URL
  - SourceLinkingManager — centralizes retrieval and external opening of an article’s URL via java.awt.Desktop
  - Article — the article that is associated with the original source
- **Important Methods:**
  getSourceLink(), openSourceLink()
- **Video Link/Timestamp:**
  - **GUI Video:** **EECS3311_GUI.mp4**
    - **Timestamp:** 0:06:31 - 0:07:11
  - **CLI Video:** **EECS3311_CLI.mp4**
    - **Timestamp:** 0:01:31 - 0:01:39

F12 — Topic Tracking:

- **Status:** Implemented
- **Design Patterns:** Observer
- **Implemented Classes/Files/Responsibilities:**
  - NewsDashboard — registers itself as an Observer on construction; “Track Topic” and “Refresh Tracked Topics (Demo)” buttons
  - AgentController — handles user requests to create, update, and manage topics. Also coordinates retrieval of news articles for the tracked topics
  - TopicRepository — stores and retrieves the topic information
  - Topic — represents the tracked topic model
  - TopicTracker — monitors the tracked topics and triggers refreshes
  - ArticleRepository — stores the newly retrieved articles

- **Important Methods:**
  trackTopic(), untrackTopic(), getTrackedTopics(), simulateTopicRefresh(), notifyObservers(), addObserver(), onNewsArticles()
- **Video Link/Timestamp:**
  - **GUI Video: EECS3311_GUI.mp4**
    - **Timestamp:** 0:12:44 - 0:13:58

F13 — Personalized Feed:

- **Status:** Implemented
- **Design Patterns:** Facade
- **Implemented Classes/Files/Responsibilities:**
  - NewsDashboard — "My Feed" button; renders the ranked articles in a scrollable dialog
  - NewsAnalysisAgent — implements the personalized feed and article scoring, weighing saved status, topic matches, tracked topics, and search history keywords
  - ArticleRepository — provides the saved articles
  - TopicRepository — provides the tracked topics
  - SearchHistoryRepository — provides the user's recent searches
  - TopicTracker — provides the tracked topic names
- **Important Methods:**
  generatePersonalizedFeed(), scoreArticle(), getAllStoredArticles(), getAllTopics(), getSearchHistory(), getTrackedTopics()
- **Video Link/Timestamp:**
  - **GUI Video: EECS3311_GUI.mp4**
    - **Timestamp:** 0:13:59 - 0:14:26

F14 — Saved Articles:

- **Status:** Implemented
- **Design Patterns:** N/A
- **Implemented Classes/Files/Responsibilities:**
  - ArticleViewer — "Save Article" button
  - NewsDashboard — "View Saved Articles" button; renders the saved list
  - ArticleRepository — stores and retrieves the user's selected articles
  - Article — the article that's being saved or removed
- **Important Methods:**
  saveArticle(), getSavedArticles(), isSaved(), setSaved(), getAllStoredArticles()
- **Video Link/Timestamp:**
  - **GUI Video: EECS3311_GUI.mp4**
    - **Timestamp:** 0:06:08 - 0:06:30

F15 — Natural-Language News Queries:

- **Status:** Implemented base code. Needs LLM model change
- **Design Patterns:** Facade, Strategy, Adapter
- **Implemented Classes/Files/Responsibilities:**
  - NewsDashboard — "Ask the AI Agent" chat panel with a question input and "Ask Agent" button
  - NewsCLI — provides a command-line interface for submitting the queries
  - AgentController — provides a simplified interface to the AI agent subsystem. Also provides the news retrieval functionality that can be used as an agent tool

    - NewsAnalysisAgent — coordinates the reasoning, planning, tool use, and response generation
    - PlanningEngine — builds a LangChain4j AiServices proxy for the NewsAssistant interface, with the tool enabled model and the NewsAgentTools instance
    - NewsAgentTools — @Tool - annotated methods the LLM can call
    - ArticleRepository — provides access to stored articles as context
    - SearchHistoryRepository — stores the user's queries for future reference
    - LLMClient — provides access to the language model
- **Important Methods:**
  answerQuery(), chat(), searchNews(), searchTechNewsRSS(), getStoredArticles(), countStoredArticles(), getChatModel(), recordSearch()
- **Video Link/Timestamp:**
    - **GUI Video:** **EECS3311_GUI.mp4**
        - **Timestamp:** 0:14:59 - 0:15:52

F16 — Daily Briefing Generation:

- **Status:** Implemented and Modified
- **Design Patterns:** Strategy, Facade
- **Implemented Classes/Files/Responsibilities:**
    - NewsDashboard — "Daily Briefing" button; opens a modeless JDialog that updates as the SwingWorker progresses
    - AgentController — provides the application-level operation for generating a briefing
    - NewsAnalysisAgent — coordinates the article retrieval, clustering, and summarization
    - ArticleRepository — provides the articles that are used to create the briefing
    - Article — the articles being briefed
    - AnalysisStrategy — a generic interface for each analysis strategy
    - Clustering — the strategy that groups related stories into event clusters for briefing
    - Summarization — the strategy that condenses each article into a summary
    - LLMClient — provides the language model processing needed for clustering and summarization
- **Important Methods:**
  generateDailyBriefing(), clusterArticles(), extractiveSummary(), getAllStoredArticles()
- **Video Link/Timestamp:**
    - **GUI Video:** **EECS3311_GUI.mp4**
        - **Timestamp:** 0:14:28 - 0:14:55

F17 — Search History:

- **Status:** Implemented
- **Design Patterns:** N/A
- **Implemented Classes/Files/Responsibilities:**
    - NewsDashboard — "Search History" button with a dialog that shows all queries and offers "Clear History"
    - AgentController — coordinates the query request
    - NewsAnalysisAgent — delegates to the repository
    - SearchHistoryRepository — stores, retrieves, and manages past queries made by the user
- **Important Methods:**
  recordSearch(), getSearchHistory(), clearSearchHistory(), save(), findAll(), clear(), fetchArticles()
- **Video Link/Timestamp:**

- **GUI Video:** **EECS3311_GUI.mp4**
  - **Timestamp:** 0:12:24 - 0:12:43

---

3. Implemented Design Patterns

3.1 Design Patterns

DP01 — Strategy:

- **Design Problem Addressed:** The agent needs to perform 6 different kinds of AI analysis on articles (i.e., summarization, categorization, entity extraction, claim extraction, comparison, and clustering) that all share the same shape, as in accepting an input and producing an output. Without abstraction, the NewsAnalysisAgent would need a long chain of if/else statements, and adding a new analysis would require modifying the agent class.
- **Participating Classes/Interfaces and Pattern Roles:**
  - **Strategy Interface:** AnalysisStrategy — declares 'R analyze(T input)'
  - **Concrete Strategies:** Summarization, Categorization, EntityExtraction, ClaimExtraction, Comparison, and Clustering
  - **Context:** NewsAnalysisAgent — holds one instance of each strategy and delegates to them
- **Relevant Source Files and Methods:**
  - AnalysisStrategy.java — analyze(T)
  - Summarization.java, Categorization.java, EntityExtraction.java, ClaimExtraction.java, Comparison.java, Clustering.java — each implements analyze()
  - NewsAnalysisAgent.java — summarize(), categorize(), extractEntities(), extractClaims(), compare(), clusterArticles()
- **Implementation and Usefulness:** Every strategy is a self contained class with a single analyze() method. The NewsAnalysisAgent instantiates each strategy in its constructor (injecting the shared LLMClient), then exposes a semantic method (summarize(Article), categorize(Article), etc.) that simply delegates to the corresponding strategy. Each strategy has no knowledge of the others or of the agent.
  This pattern is useful because adding a seventh analysis would only require one new class and one new delegating method in NewsAnalysisAgent. All the already existing strategies remain untouched. As well, each strategy is independently testable.
- **Stage 1 Comparison:** This design pattern matches my Stage 1 exactly.

DP02 — Observer:

- **Design Problem Addressed:** When a tracked topic receives new articles in the background, the GUI must update without polling. The tracker in the agent layer should not know which UI components are interested. Otherwise, the agent layer would depend on Swing classes, which violates the MVC design pattern established in Stage 1.
- **Participating Classes/Interfaces and Pattern Roles:**
  - **Subject:** TopicTracker — maintains a list of tracked topics and a list of registered 'Observer S'
  - **Observer Interface:** TopicTracker.Observer — declares 'onNewArticles(String topic, int count)'
  - **Concrete Observer:** NewsDashboard — registers an observer in its constructor that shows a JOptionPane popup when notified

- **Relevant Source Files and Methods:**
    - TopicTracker.java — addObserver(), trackTopic(), untrackTopic(), getTrackedTopics(), notifyObservers()
    - NewsAnalysisAgent.java — simulateTopicRefresh() calls notifyObservers() after fetching news articles for each tracked topic
    - AgentController — exposes getTopicTracer() to the view
- **Implementation and Usefulness:** The TopicTracker is completely unaware of Swing. It only knows that some object implements the Observer interface. When 'NewsAnalysisAgent.simulateTopicRefresh()' finds new articles for a tracked topic, it calls 'notifyObserver(topic, count)' which iterates over the registered observers, and invokes onNewArticles. The observer registered in the dashboard will then post a Swing popup.

  This pattern is useful because multiple UI surfaces can subscribe without the tracker changing, the tracker is testable without a GUI, and also the agent layer remains free of Swing dependencies.
- **Stage 1 Comparison:** This design pattern matches with my Stage 1, though the implementation is slightly different. Instead of using the automatic background 'java.swing.Timer' code, a button is used instead to trigger 'simulateTopicRefresh()' manually.

DP03 — Factory Method:

- **Design Problem Addressed:** The system must support multiple news sources (NewsAPI and RSS) whose adapters have different constructors. If the NewsAnalysisAgent instantiated adapters directly, they would need to know every adapter type and the feed routing logic, coupling the agent layer to the source layer.
- **Participating Classes/Interfaces and Pattern Roles:**
    - **Factory:** NewsSourceFactory — static methods createAdapter(String type) and createAdapter(String type, String query)
    - **Product Interface:** NewsSourceAdapter — declares fetchArticles(String query)
    - **Concrete Products:** RSSAdapter, NewsAPIAdapter
- **Relevant Source Files and Methods:**
    - NewsSourceFactory.java — createAdapter(String), createAdapter(String, String), pickFeedForQuery(String), getCategoryHint(), isKnownCategory(String)
    - NewsAnalysisAgent.java — retrieveArticles(String sourceType, String query) calls NewsSourceFactory.createAdapter(sourceType, query)
- **Implementation and Usefulness:** The agent layer only ever sees the NewsSourceAdapter interface. It passes a string identifier from the GUI button, and the user's query to the factory. The factory then returns the correct concrete adapter. Adding a third source requires only one new class and one new branch in the factory. Nothing else will change.

  This pattern is useful because the agent layer and the GUI stay decoupled from adapter construction, and from feed routing decisions. It also makes the factory independently testable where you can verify the correct adapter is returned for a given source type without ever touching the network.
- **Stage 1 Comparison:** This design pattern matches with my Stage 1 exactly.

DP04 — Adapter:

- **Design Problem Addressed:**
- **Participating Classes/Interfaces and Pattern Roles:**
    - **Target Interface:** NewsSourceAdapter — declares 'List<Article> fetchArticles(String query)'
    - **Adapters (Concrete):** RSSAdapter and NewsAPIAdapter

    - **Adaptee (NewsAPI Side):** Jackson's JsonNode plus Jsoup's Document
    - **Adaptee (RSS Side):** Rome's SynFeed / SynEntry plus Jsoup's Document
    - **Domain Target:** Article
- **Relevant Source Files and Methods:**
    - NewsAPIAdapter.java — fetchArticles(), scrapeArticle()
    - RSSAdapter.java — fetchArticles(), scrapeArticle(), truncate()
    - Article.java — the target domain type
- **Implementation and Usefulness:** Each provider performs 3 translations: 1) Provide format parsing — RSS is parsed via Rome and NewsAPI is parsed via Jackson. Both produce provider specific intermediate objects (SyndEntry, JsonNode) that never leave the adapter; 2) Full text extraction — both adapters use Jsoup to fetch the original URL and extract the article adapter, though each adapter encapsulates its own approach; 3) Domain construction — both adapters construct Article objects with a normalized field set (id, title, content, url, date). The AgentController, NewsAnalysisAgent, repositories, and GUI only ever see Article.

  This pattern is useful because the domain and analysis layers only know about Article. Adding a third provider would require only a new adapter class implementing NewsSourceAdapter. There would be no changes to the agent, repositories, or GUI. As well, it lets each adapter encode provider specific functionalities without them leaking into the rest of the codebase.
- **Stage 1 Comparison:** This design pattern matches with my Stage 1 exactly.

DP05 — Facade:

- **Design Problem Addressed:** The GUI and CLI need to invoke a wide range of operations (i.e., fetching, analysis, comparison, briefing, topic management, search history, personalized feeds, timeline generation, source linking, natural-language queries, and topic tracking). Implementing all of that directly would force every UI class to know about the NewsAnalysisAgent, LLMClient, PlanningEngine, NewsAgentTools, SourceLinkManager, TopicTracker, and the three repositories. The wiring would be fragile and duplicated across Swing views.
- **Participating Classes/Interfaces and Pattern Roles:**
    - **Facade:** AgentController
    - **Subsystems Behind the Facade:**
        - NewsAnalysisAgent (core agent)
        - LLMClient
        - PlanningEngine and NewsAgentTools
        - SourceLinkManager
        - TopicTracker
        - ArticleRepository, TopicRepository, SearchHistoryRepository
- **Relevant Source Files and Methods:**
    - AgentController.java:
        - **Retrieval and analysis:** fetchNews(), summarize(), categorize(), extractEntities(), extractClaims()
        - **Multiple articles:** compareArticles(), clusterStories()
        - Timeline, briefing, feed: generateTimeline(), generateDailyBriefing(), generatePersonalizedFeed()
        - **Topic management:** createTopic(), getAllTopics(), deleteTopic()

        - **Search history:** recordSearch(), getSearchHistory(), clearSearchHistory()
        - Topic tracking: trackTopic(), untrackTopic(), getTrackedTopics(), getTopicTracker(), simulateTopicRefresh()
        - **Source linking:** getSourceLink(), openSourceLink()
        - Natural-language: answerQuery()
        - **Article storage:** getAllStoredArticles(), saveArticle(), getSavedArticles()
- **Implementation and Usefulness:** The controller constructs the entire agent subsystem in its constructor (LLM client, repositories, agent, source link manager) and holds them privately. Each public method is a simple delegator. For example, 'summarize(Article a)' simply calls 'agent.summarize(a)'. Now, the GUI only ever talks to AgentController. It never touches NewsAnalysisAgent, LLMClient, or any repository directly. For example, the ComparisonPanel only needs the three facade calls, 'controller.clusterStories()', 'controller.compareArticles(...)', and 'controller.openSourceLink(...)'.
  This pattern is useful because the GUI and CLI share the exact same facade, so business logic is only written once. If the internal agent architecture changes (e.g., you swap our PlanningEngine), only the facade needs to be updated. Every view will continue to work. As well, it substantially shortens the classes that use it.
- **Stage 1 Comparison:** This design pattern matches with my Stage 1, with the only difference being that my Stage 1 facade delegation is slightly broader.

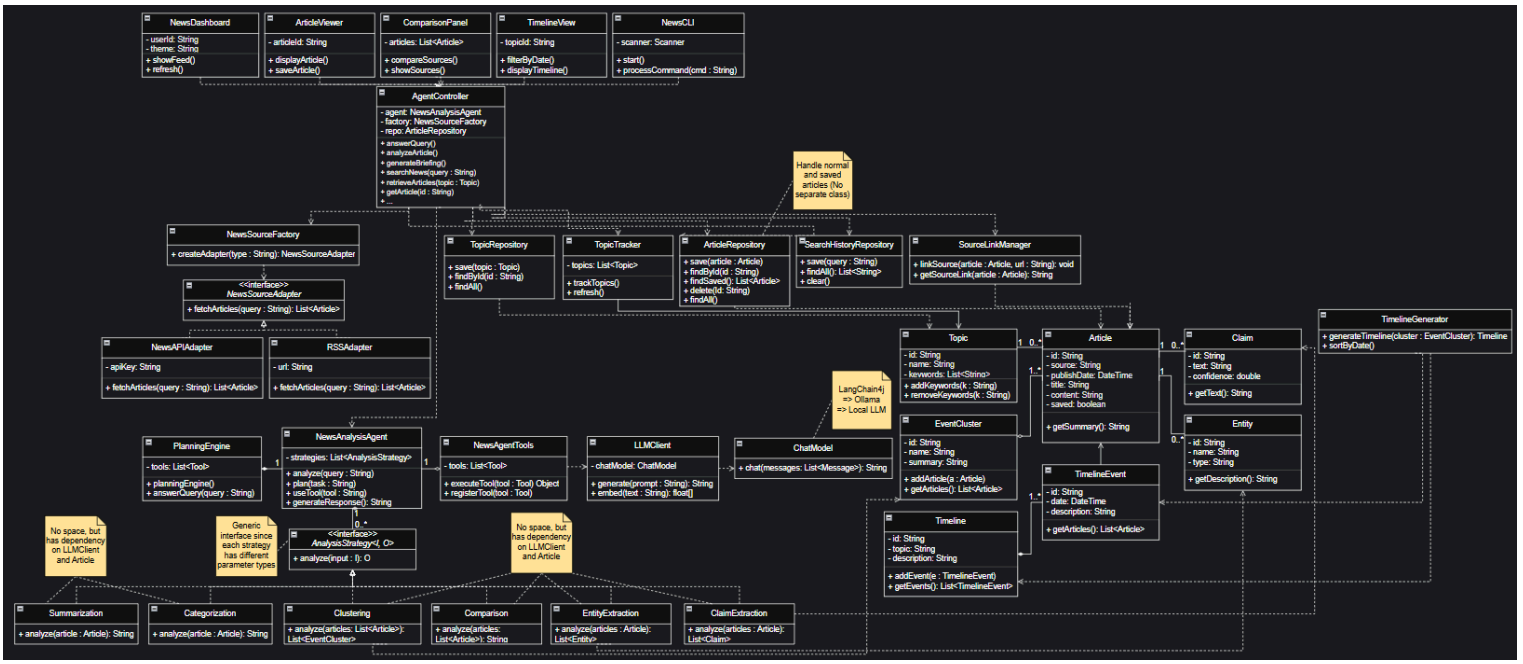
DP06 — MVC:

- **Design Problem Addressed:** The system has both a GUI and a CLI that must share the same logic and data. Without separation, the Swing code and analysis logic would become tangled. This would make it untestable since it is duplicated in the CLI and impossible to reuse across views.
- **Participating Classes/Interfaces and Pattern Roles:**
    - **Model:** Article, Topic, Timeline, TimelineEvent, Claim, Entity — domain state and rules
    - **View:** NewsDashboard, ArticleViewer, ComparisonPanel — Swing views that render models and forward user actions
    - **Controller:** AgentController
- **Relevant Source Files and Methods:**
    - **Model:** Article.java, Topic.java, Timeline.java, TimelineEvent.java, Entity.java, Claim.java
    - **View:** NewsDashboard.java, ArticleViewer.java, ComparisonPanel.java
    - **Controller:** AgentController.java — every view action routes through a facade method. The controller updates the models via the agent layer and returns the results to the view for rendering.
    - **Entry point:** Main.java — creates the controller once and launches either GUI or the CLI.
- **Implementation and Usefulness:** The implementation follows the following pattern: 1) Models are pure data — Article, Topic, Timeline, TimelineEvent, Entity, and Claim contain only fields and accessors so that they can be reused by any front end; 2) Views only talk to the controller — every Swing listener in NewsDashboard, ArticleViewer, and ComparisonPanel invokes a method on AgentController and renders the returned data. No view will import NewsAnalysisAgent, LLMClient, PlanningEngine, or any repository. Meaning LLM calls run inside SwingWorker so that the views remain responsive; 3) The controller holds the domain logic — AgentController owns the NewsAnalysisAgent, LLMClient, SourceLinkManager, and repositories. Meaning, it coordinates input (query, article) → service (agent method) → model updates → view display.
  This pattern is useful because all the analysis logic lives behind the controller. The CLI can reuse the entire stack with only a different Main entry path. As well, models are testable without any UI. Adding a new view

requires only a new class that talks to the same controller. No changes are needed in the agent, strategies, repositories, and sources.

- **Stage 1 Comparison:** This design pattern matches my Stage 1 with a few minor differences in the implemented classes. All features still work and there was no structural change, but I did do a scope reduction where not all classes were implemented (e.g., EventCluster, EventClusterRepository, TimelineView, NewsController, TopicController, and FeedController).

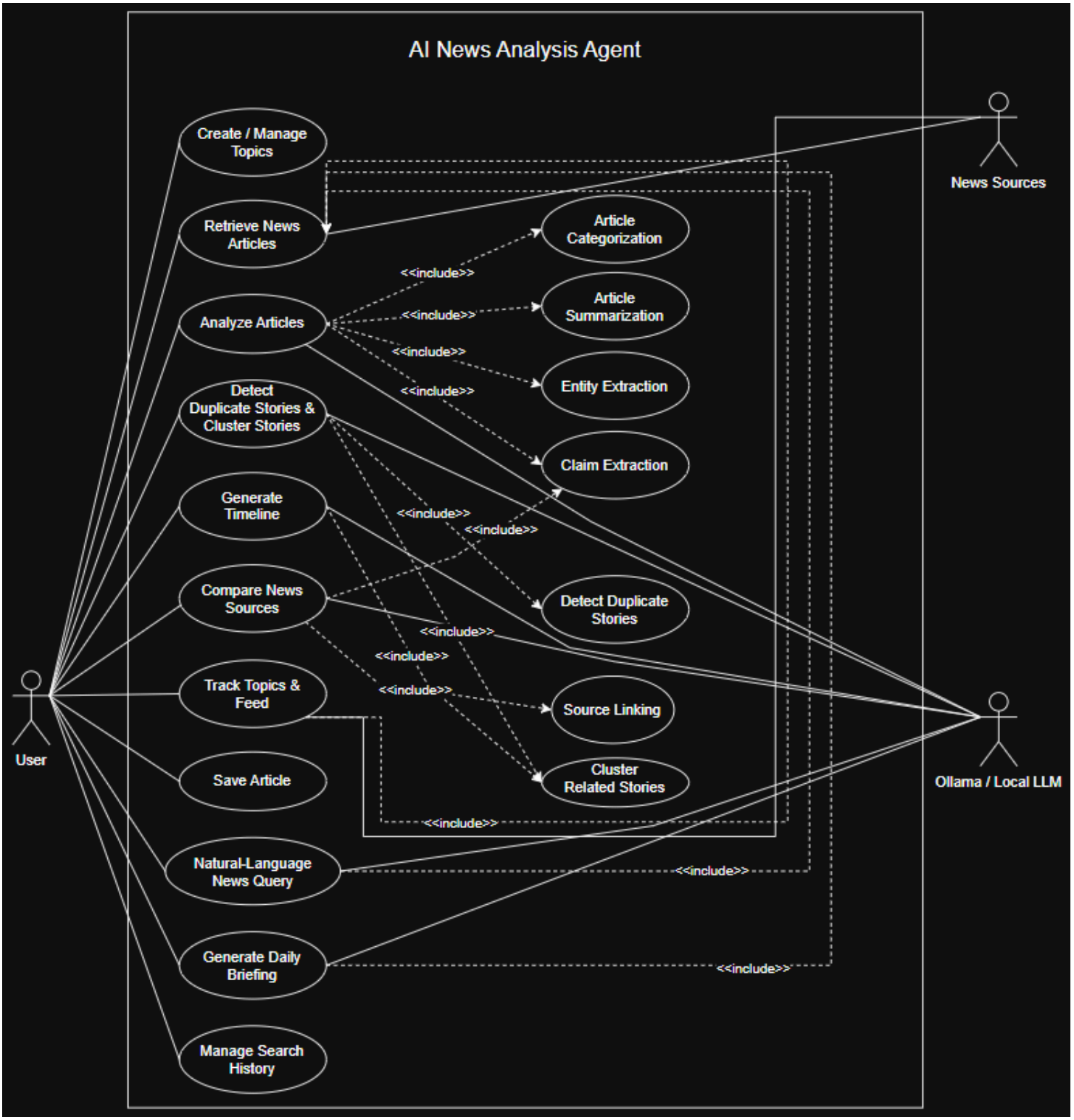
3.2 Updated UML Class Diagram



Link to Diagram: https://drive.google.com/file/d/1ilQV5C88PpkW4x3YMEVa2ogBW8Szfg6G/view?usp=sharing

3.3 Updated Use-Case Diagram

The Use-Case Diagram remains the same. All features were implemented to the best of my abilities.



Link to Diagram: https://drive.google.com/file/d/1z4CGwGN0o2oePlOG1IictK7H_20-9j4X/view?usp=sharing

3.4 Updated Sequence Diagrams

Important Note:

Same as in Stage 1, due to time, the graphical representation of a UML sequence diagram is made for only four of the eight as examples of how the text-based sequence will be translated over.

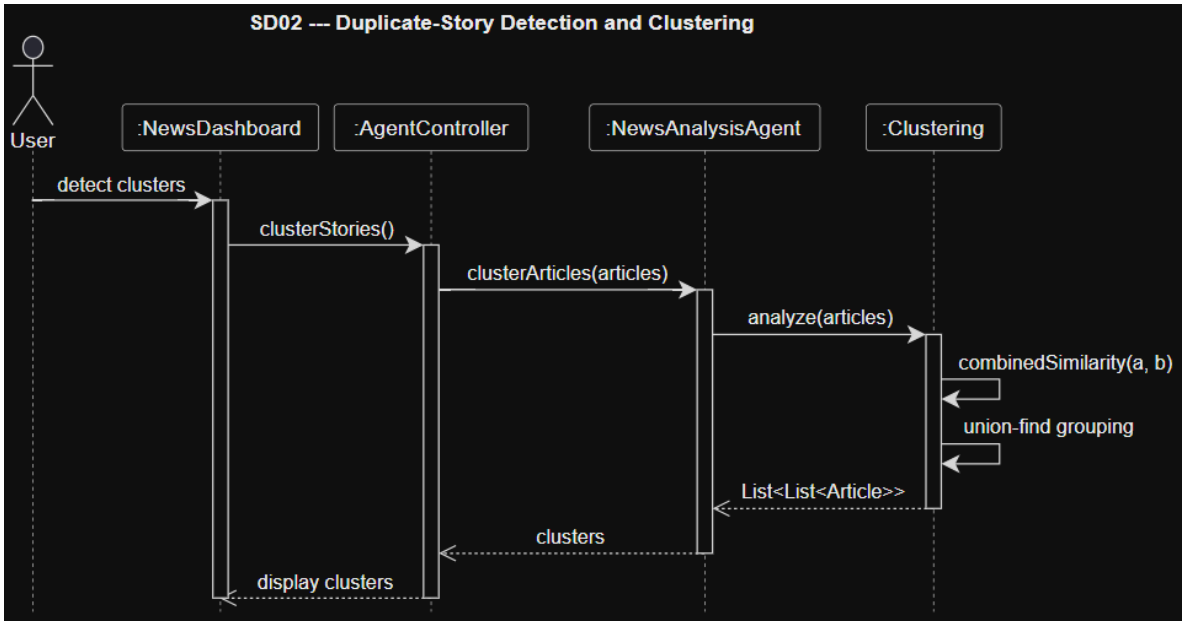
SD01 — Article Retrieval and Analyzation:

- User → NewsDashboard → AgentController → NewsAnalysisAgent → NewsSourceFactory → NewsSourceAdapter (RSSAdapter | NewsAPIAdapter) → External source (NewsAPI.org / BBC RSS / Google News) → Article objects → ArticleRepository → NewsAnalysisAgent → AgentController → NewsDashboard

- User → ArticleViewer → AgentController → NewsAnalysisAgent → AnalysisStrategy → LLMClient → OllamaChatModel → Ollama → Local LLM → Result string → ArticleViewer

SD02 — Duplicate-Story Detection and Cluster Events:

- User → NewsDashboard → AgentController → NewsAnalysisAgent → ArticleRepository → Clustering → NewsAnalysisAgent → AgentController → NewsDashboard



SD03 — Multiple Source Comparison and Source Linking:

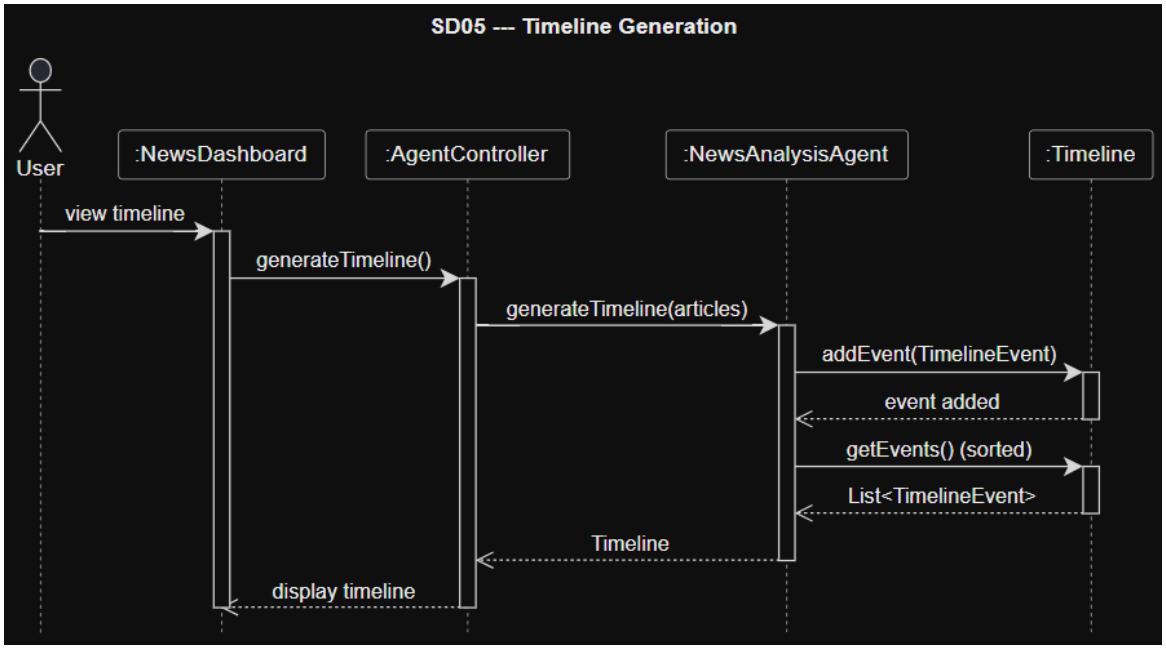
- User → NewsDashboard → ComparisonPanel → AgentController → NewsAnalysisAgent → Clustering → ComparisonPanel → AgentController → NewsAnalysisAgent → Comparison → LLMClient → OllamaChatModel → Ollama → Local LLM → Comparison result → ComparisonPanel
- User → ComparisonPanel → AgentController → SourceLinkManager → java.awt.Desktop.browse(URL) → System browser

SD04 — Topic Tracking and Feed Updates:

- User → NewsDashboard → AgentController → NewsAnalysisAgent → TopicRespository
- User → NewsDashboard → AgentController → NewsAnalysisAgent → TopicTracker → TopicRepository
- User → NewsDashboard → AgentController → NewsAnalysisAgent → NewsSourceFactory → NewsSourceAdapter → ArticleRepository → TopicTracker → NewsDashboard

SD05 — Timeline Generation:

- User → NewsDashboard → AgentController → NewsAnalysisAgent → ArticleRepository → Timeline → NewsAnalysisAgent → AgentController → NewsDashboard

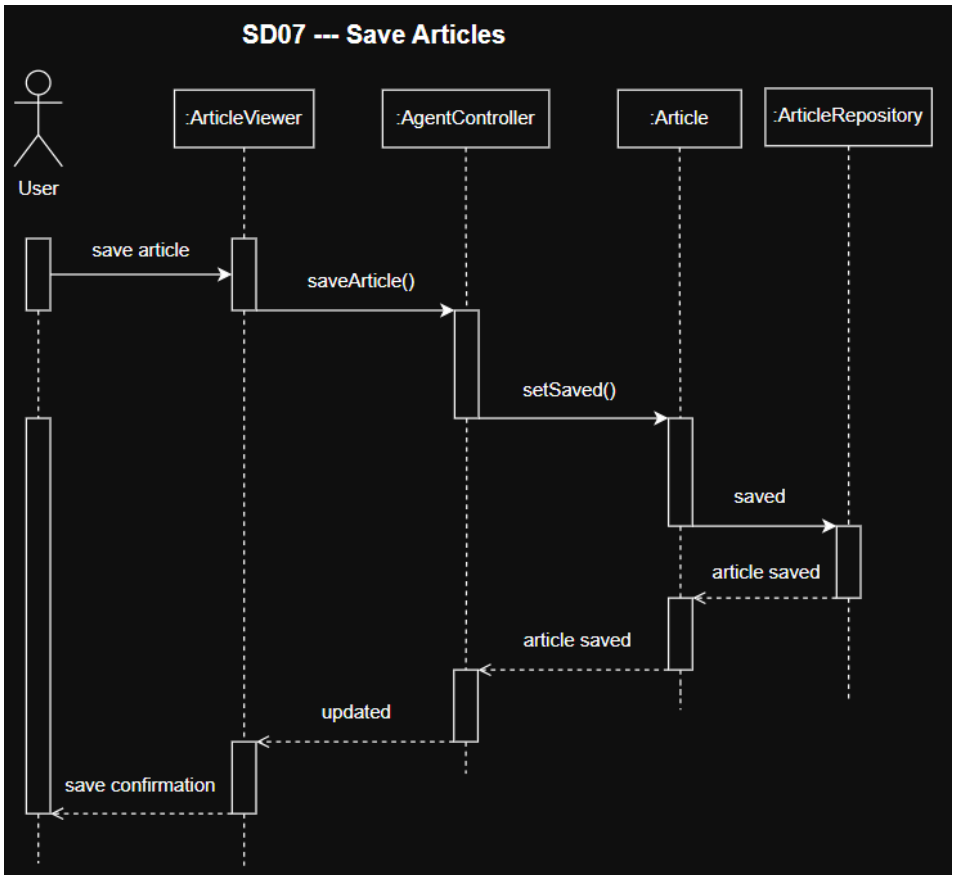


SD06 — Natural-Language News Queries:

- User → NewsDashboard → AgentController → NewsAnalysisAgent → SearchHistoryRepository → PlanningEngine → NewsAgentTools → NewsSourceFactory → NewsSourceAdapter (NewsAPIAdapter | RSSAdapter) → ArticleRepository → LLMClient → OllamaChatModel → Ollama → Local LLM → Response → PlanningEngine → NewsAnalysisAgent → AgentController → NewsDashboard

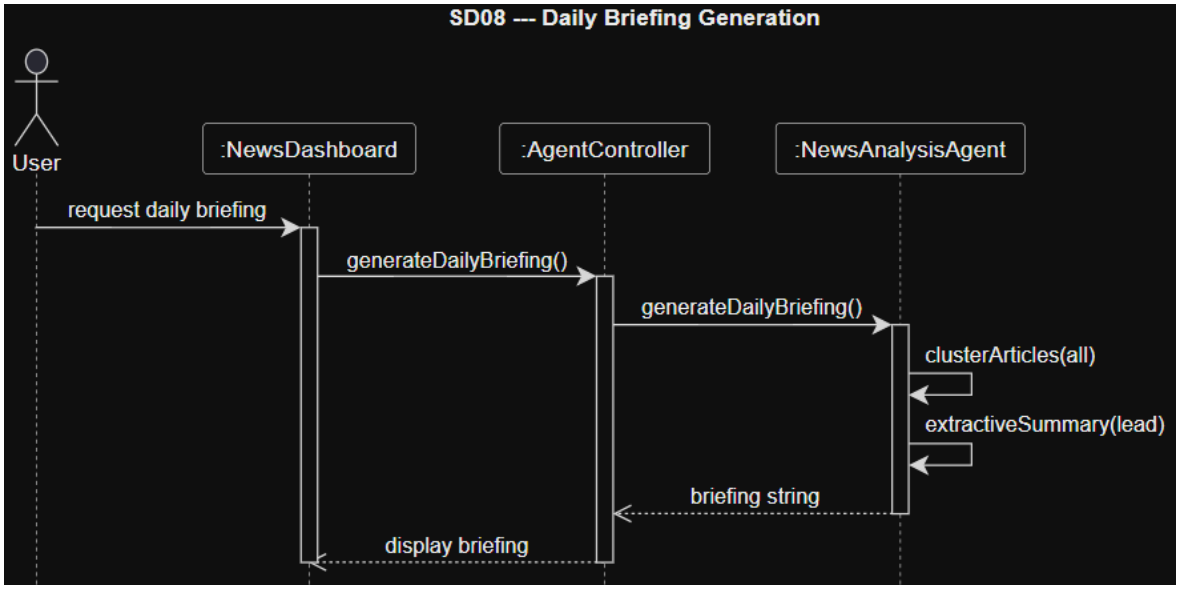
SD07 — Saved Articles:

- User → ArticleViewer → AgentController → Article → ArticleRepository



SD08 — Daily Briefing Generation:

- User → NewsDashboard → AgentController → NewsAnalysisAgent → ArticleRepository → Clustering → Briefing string → NewsAnalysisAgent → AgentController → NewsDashboard



Link to Diagrams: https://drive.google.com/file/d/19gzBpxHhZbtYoKVNZetNjoMfyEr46BQx/view?usp=sharing

---

4. Changes and Rationale

4.1 Architectural and Class Changes

Original Design	Implemented Change	Reason	Impact
Multiple Controllers: NewsController, TopicController, FeedController, and AgentController as separate classes.	**Single Controller:** AgentController now acts as the sole facade and controller for both the GUI and CLI.	Reducing the number of classes kept the codebase maintainable and easier to track. As well, it eliminated unnecessary boilerplate and wiring between different controllers.	It simplifies the MVC architecture. The GUI (NewsDashboard, ArticleViewer, and ComparisonPanel) only needs to interact with one class. All the controller responsibilities are centralized which makes it easier to debug and extend if needed.
FeedGenerator class dedicated to generating the personalized feed.	**Consolidated Logic:** The feed generation logic was moved directly into NewsAnalysisAgent (generatePersonalizedFeed() and scoreArticle()).	Since FeedController was removed, a standalone generator became redundant. The agent already had access to the TopicRepository and SearchHistoryRepository that was needed for scoring.	Removes two classes (FeedController and FeedGenerator). The scoring logic is now shared with the agent's other analysis strategies which reduces coupling.
TimelineView and TimelineGenerator classes dedicated to generating the timeline.	**Inline Rendering:** Timeline generation logic now lives in NewsAnalysisAgent (generateTimeline()), and rendering now happens in a scrollable JOptionPane	Having a separate view class was overkill for a simple chronologically sorted text list, so I removed it.	It reduced boilerplate whilst still preserving the MVC pattern (the controller still returns a Timeline model to the View for rendering).

	dialog inside NewsDashboard		
EventCluster and EventClusterRepository models dedicated for handling the clustering strategy.	**Direct Lists:** The clustering results are returned and handled as List<List<Article>>, and I removed the repository for clusters.	Clusters last for a very short time and are recalculated on demand. So by persisting them to a repository, it added unnecessary state management and increased the complexity.	It simplified the state handling. The Clustering strategy remains a pure function (Input: List<Article>, Output: List<List<Article>>), and makes it very easily testable.

4.2 Feature Modifications

Original Design	Implemented Change	Reason	Impact
F16 (Daily Briefing Generation): Use the LLM to summarize all articles within each cluster.	**Extractive summarization:** The LLM call was removed. The system extracts the first 1–2 meaningful sentences of each cluster's lead article using pure Java string manipulation (extractiveSummary()).	The original design required processing thousands of characters across multiple articles. This caused the LLM to either time out, or take several minutes to form an output.	The daily briefing now generates in under a second. **Trade-off:** The summary is less abstractive and detailed compared to an LLM generated one.
F05 (duplicate-story detection) & F06 (event clustering): Use LLM embeddings to compute similarity and group articles.	**Deterministic Jaccard Similarity:** The LLM is no longer called. Clustering.analyze() now uses a weighted Jaccard similarity matrix (60% title, 40% content preview), union-find for grouping the articles.	Calling the local LLM to generate embeddings for every article, and then computing the similarity was taking way too long.	The clustering feature now works in under a second. As well, upon a skim through the output, it still effectively identifies duplicate stories.

4.3 Algorithmic and Pattern Changes

Original Design	Implemented Change	Reason	Impact
F15 (Natural-Language Queries): Use a custom PlanningEngine and ToolManager to route queries and call tools.	**LangChain4J AiServices:** The custom ToolManager was removed. The PlanningEngine now uses LangChain4j's AiServices.builder() to bind NewsAgentTools directly to the LLM.	LangChain4j natively handles tool calling, planning, and multi-step execution. Reinventing this with a custom ToolManager would have been redundant and more prone to error.	In theory, after a model change, the LLM should now autonomously decide which tools to invoke (searchNews, searchTechRSS, or getStoredArticles) based on the system prompt. This would make the agent's behaviour more robust.
F02 (Article Retrieval): Adapters simply parse the provider's response (JSON or RSS XML).	**Jsoup Scraping:** Both NewsAPIAdapter and RSSAdapter now attempt to fetch the original URL and scrape the full article text, falling back to the provider's description if scraping is blocked.	NewsAPI and RSS often return truncated snippets (e.g., 1-2 sentences). My LLM strategies (F03, F04, and F07) need the full article text in order to produce meaningful summaries and extractions.	It greatly improved the quality of the AI analysis. **Trade-off:** It increases the fetch time slightly per article, and causes the LLM's analysis speed to be greatly increased as well.
DP02 (Observer): Background	**Manual Trigger:** The timer is commented out. Instead,	Automatically polling NewsAPI every 2 minutes	It preserves the Observer pattern implementation

javax.swing.Timer triggers topic refresh automatically.	the user triggers the updates via a “Refreshed Tracked Topics (Demo)” button.	rapidly exhausted the 100 request daily limit on the free tier during my development and testing.	(the GUI still registers and receives callbacks), but avoids quota exhaustion. This design also supports re-enabling the timer in a production use scenario.

---

5. AI Tools and Collaboration Log

Tool/Model Used: Deepseek

Purpose: Code generation, and debugging

Example Tasks:

1. **Task:** Since this is my first time working AI into an application, I am trying to figure out how you would integrate and use LangChain4j, Ollama, and llama3 in a Java project.
    - **Input:** How do you go about setting up LangChain4j, Ollama, and llama3 in a Java project?
    - **AI Contribution:**
        - Setting this up involves three main parts: preparing Ollama with a Llama model locally, adding the LangChain4j dependencies to your Java project, and then writing the code to connect them.
        - **1)** Download Ollama: go to ollama.com and download the installer for your operating system (Windows/macOS/Linux); **2)** Verify Installation: after installation is complete, open a terminal and run ‘ollama –version’. If a version number is displayed, the installation was successful; **3)** Download the llama3 Model: run ‘ollama run llama3’ in the terminal; **4)** Test the Model: type a few words at the >>> prompt. If you get a response, the model is ready.
        - **Example Code:**

```
class OllamaChatLocalModelTest {
  static String MODEL_NAME = "llama3.2"; // try other local ollama model names
  static String BASE_URL = "http://localhost:11434"; // local ollama base url
  public static void main(String[] args) {
     ChatModel model = OllamaChatModel.builder()
           .baseUrl(BASE_URL)
           .modelName(MODEL_NAME)
           .temperature(0.8)
           .build();
     String answer = model.chat("List top 10 cites in China");
     System.out.println(answer);
   }
}
```

    - **Human Contribution / Outcome:**
      I followed the installation process above, and made sure everything was running successfully. I then used the example code to aid in creating my LLMClient class, and made sure to modify the parameters to match the model I chose (llama3). In addition to this, I added a generate function that allows the user to generate a prompt. All of this was then tested in my Main class.

2. **Task:** I get the error message that the constructor, 'XmlReader(url)' is deprecated, and I am trying to fix this.
    - **Input:** I am currently coding an RSSAdapter class for fetching news articles. In the piece of code below, I'm trying to open the feed URL and build a SyndFeed object with ROME. However, there is a line through 'XmlReader(url)' and when I hover it, it tells me that the constructor is deprecated. Is there an alternative method I could use to fix this?

```
URL url = new URL(feedUrl);
SyndFeedInput input = new SyndFeedInput();
SyndFeed feed = input.build(new XmlReader(url));
```

    - **AI Contribution:**
        - That warning is completely normal! It just means the rome library author marked that specific constructor as "deprecated" (meaning they prefer you use a newer way of doing it in future versions), but the code will still work perfectly fine.
        - If you want to get rid of the strike-through and use the modern, non-deprecated approach, you can change the new XmlReader(url) to open an InputStream first.
        - **Example Code:**

```
// Use URI and openStream() to avoid the deprecated XmlReader(URL) constructor
URI uri = URI.create(feedUrl);
InputStream stream = uri.toURL().openStream();
SyndFeedInput input = new SyndFeedInput();
SyndFeed feed = input.build(new XmlReader(stream));
```

    - **Human Contribution / Outcome:**
      I replaced my snippet of code with the example code above, and added the correct imports. This removed the deprecated message and works perfectly.
3. **Task:** I am trying to figure out a good way to compare the similarity of two sets of strings for my news analysis comparison strategy.
    - **Input:** I'm trying to calculate the similarity between two strings based on their distinct word sets (case sensitive), for my comparison method in Java. What is a good way to implement this?
    - **AI Contribution:**
        - A common approach is the Jaccard similarity coefficient: the size of the intersection divided by the size of the union of two sets. It ranges from 0 (no shared words) to 1 (identical word sets).
        - **Example Code:**

```
import java.util.*;
public class WordSetSimilarity {
   public static double jaccardSimilarity(String a, String b) {
      Set<String> setA = distinctWords(a);
      Set<String> setB = distinctWords(b);
      // Both empty: define as 1.0 (identical) or 0.0 depending on your needs
      if (setA.isEmpty() && setB.isEmpty()) {
         return 1.0;
      }
      // Iterate over the smaller set for efficiency
      Set<String> smaller = setA.size() <= setB.size() ? setA : setB;
      Set<String> larger  = smaller == setA ? setB : setA;
```

```
        int intersection = 0;
        for (String word : smaller) {
            if (larger.contains(word)) {
                intersection++;
            }
        }
        int union = setA.size() + setB.size() - intersection;
        return (double) intersection / union;
    }
    private static Set<String> distinctWords(String s) {
        // Split on any run of non-word characters; keeps case sensitivity
        // Adjust the regex for your definition of a "word"
        return new HashSet<>(Arrays.asList(s.split("\\W+")));
    }
}
```

- **Human Contribution / Outcome:**
  I followed the example code provided, and updated it to fit my needs for my comparison strategy.

4. **Task:** I am having issues with displaying full article content, which is affecting how my news analysis strategies are working. I am trying to fix this.
   - **Input:** Here is my code for RSSAdapter and NewsAPIAdapter: (‘code here’). When I click the view button after selecting an article, I want the entire article content to be displayed in the center panel. Currently though, it only displays a single line description of the article. This is affecting my news analysis strategies from operating correctly. How can I fix this?
   - **AI Contribution:**
     - This is because of a limitation of the NewsAI’s free tier, not your code. To fix this, you can scrape the full article with the ‘jsoup’ Java library.
     - Caveat: Some websites may block web scraping, so ensure that you have a fallback to your original snippet of code capturing only the description as the content.
   - **Human Contribution / Outcome:**
     I followed the recommendation and used ‘jsoup’ in my RSSAdapter and NewsAPIAdapter to scrape the entirety of the content. This worked out perfectly, and the full article is now displayed in my NewsDashboard panel.
5. **Task:** My news analysis features keep timing out, and I am trying to fix this.
   - **Input:** Here is my code for RSSAdapter and NewsAPIAdapter: (‘code here’). After changing both to use ‘jsoup’ for scraping the content, I now get the entirety of the article displayed in my panel. The issue I am having now is that my analysis features keep timing out since the LLM is taking too long. What can I do to fix this?
   - **AI Contribution:**
     - Possible solutions:
       - Option 1: Increase the timeout threshold of your LLM client
       - Option 2: Truncate the article’s content before using it for analysis
   - **Human Contribution / Outcome:**
     I implemented the ‘Options 2’ suggestion above, adding truncation to the content before calling the

LLM to analyze it. I have set the maximum number of characters in the content to be 3000. Anything above is left out. While still a little slow, this seems to work out and doesn't cause a timeout error on my machine.

6. **Task:** My 'comparison' and 'detect duplicate/cluster' strategies work for only one news fetcher (i.e., NewsAPI and RSSAdapter) during one run time.
    - **Input:** Insert code: ('code here'). If I fetch news from RSSAdapter first, followed by using the 'detect duplicate stories/clusters' strategy or 'comparison' strategy, these two strategies will then only work for any other future RSSAdapter news fetching. That is, if I then enter a topic and fetch news from NewsAPI, these two strategies will not work for any of the fetched NewsAPI articles and vice versa.
    - **AI Contribution:**
        - This is a real bug, and it's caused by the repositories shared across everything combined with the clustering picking up all stored articles including singletons that dominate the cluster calculation.
        - When you fetch RSS, the retrieved RSS articles are all saved to ArticleRepository. When you fetch NewsAPI, the NewsAPI articles are all saved to the SAME repository. Then when you select your 'detect duplicate stories/clusters' strategy, that returns ALL (RSS + NewsAPI mixed together).
        - Now here is the failure:
            - Root Cause #1: Different content styles make clustering fail across sources
            - Root Cause #2: RSS-only clustering is accidentally "working" because there are so few articles
            - Root Cause #3: Comparison only sends singleton articles
        - Fix:
            - Make your clustering strategy account for mixed-source content by updating your CombinedSimilarity function with the following added:
              double hostBonus = sameHost(a.getSourceUrl(), b.getSourceUrl()) ? 0.05 : 0.0;
              return (titleSim * 0.4) + (contentSim * 0.6) + hostBonus;
            - Update your clustering algorithm to be fully order-independent, such that the order of which source you fetch news from does not affect your clustering strategy from working:
                - Step 1: compute pairwise similarity matrix
                - Step 2: build connected components (union-find) using threshold
                - Step 3: group articles by component root
    - **Human Contribution / Outcome:**
      I added the two snippets of code from the first fix into my CombinedSimilarity function, then redid my clustering algorithm following the provided steps. This allowed me to cluster stories without the source and its order of selection affecting its functionality.

---

6. Development and verification summary

1.1 Weekly Commit Summary

Week 1 (Oct. 20 - 26):

- **Commit 1:** Setup Maven project and application structure to integrate LangChain4j and Ollama
- **Commit 2:** Installed and configured Ollama (llama3). Tested the LLM connection
- **Commit 3:** Implemented domain models Article and Topic
- **Commit 4:** Implemented domain models Entity and Claim
- **Commit 5:** Implemented the repositories for Article and Topic

Week 2 (Oct. 27 - Nov. 2):

- **Commit 1:** Implemented and tested AnalysisStrategy interface and Summarization strategy
    - **Issue:** The summarized result generated by the LLM was too long and simply reworded each line of the mock article into the same number of lines.
    - **Fix:** I lowered the Llama3 temperature value for better accuracy in the summarization (i.e., 0.7 ⇒ 0.5). As well, I added more detail to the summarization prompt to ensure the summarized result is not simply a paraphrase of the original article (i.e., same length, just reworded).
- **Commit 2:** Implemented the NewsSourceAdapter and RSSAdapter
- **Commit 3:** Implemented NewsAPIAdapter and NewsSourceFactory with .env protection
- **Commit 4:** Implemented AgentController and NewsAnalysisAgent. Tested the article retrieval and summarization functionalities
- **Commit 5:** Implemented a basic GUI/CLI using Java Swing GUI toolkit that wraps around the current functionalities

Week 3 (Nov. 3 - 9):

- **Commit 1:** Implemented the news analysis strategies, Categorization and EntityExtraction
- **Commit 2:** Implemented the news analysis strategies, ClaimExtraction and Comparison
- **Commit 3:** Implemented Clustering strategy. Updated NewsAnalysisAgent and AgentController to include newly added strategies
- **Commit 4:** Updated the GUI to include new strategies. Fixed the issue with fetching the entirety of an article's content
    - **Issue:** My fetched article's content was only displaying a one line description, and my news analysis strategies were only processing this one line. I used 'jsoup' to fix this, but it caused another issue where the content was now too long and would cause the LLM to time out.
    - **Fix:** I truncated the article content to be a maximum of 3000 characters before being used for analysis.
- **Commit 5:** Implemented timeline generation with Timeline and TimelineEvent models
- **Commit 6:** Implemented search history tracking
- **Commit 7:** Implemented natural-language news query
- **Commit 8:** Implemented topic creation/management and topic tracker functionalities
- **Commit 9:** Implemented daily briefing generation
- **Commit 10:** Implemented personalized feed
- **Commit 11:** Implemented multi-source comparison and source-linking
- **Commit 12:** Added text truncation in RSSAdapter. Updated RSS retrieval to work with search field

- **Commit 13:** Fixed duplicate-story clustering strategy to be less strict and more effective. Fixed comparison strategy to avoid timeout error
    - **Issue:** Currently, articles are being clustered by comparing only the title. If 50% of a title shares the same words with another, it is considered a duplicate and is clustered together. This does not work well. In fact, most of the time it just outputs that there are no matches.
    - **Fix:** Kept title comparison, but lowered threshold requirement. Added content similarity by taking a sample of characters (content) from each article. Added a check for keyword overlap. Updated the comparison prompt to have a fixed output cap to avoid timeout.
- **Commit 14:** Clustering fix across mixed RSS/NewsAPI sources
- **Commit 15:** Created the README

---

7. Demonstration index

Feature ID	Feature Description	Type	Video(s) & Timestamp
F01	Topic Creation	D	• **GUI Video:** **EECS3311_GUI.mp4** ○ **Timestamp:** 0:00:25 - 0:01:26
F02	Article Retrieval	D	• **GUI Video:** **EECS3311_GUI.mp4** ○ **Timestamp:** 0:01:28 - 0:02:32 • **CLI Video:** **EECS3311_CLI.mp4** ○ **Timestamp:** 0:00:49 - 0:01:30, 0:03:43 - 0:04:02
F03	Article Categorization	AI	• **GUI Video:** **EECS3311_GUI.mp4** ○ **Timestamp:** 0:03:57 - 0:04:25 (output at 0:04:18)
F04	Article Summarization	AI	• **GUI Video:** **EECS3311_GUI.mp4** ○ **Timestamp:** 0:03:12 - 0:03:56 (output at 0:03:48) • **CLI Video:** **EECS3311_CLI.mp4** ○ **Timestamp:** 0:01:42 - 0:03:42 (output at 0:03:34)
F05	Duplicate Detection	D	• **GUI Video:** **EECS3311_GUI.mp4** ○ **Timestamp:** 0:07:13 - 0:07:38
F06	Event Clustering	AI	• **GUI Video:** **EECS3311_GUI.mp4** ○ **Timestamp:** 0:07:13 - 0:07:38
F07	Entity Extraction	AI	• **GUI Video:** **EECS3311_GUI.mp4** ○ **Timestamp:** 0:04:26 - 0:05:06 (output at 0:04:56)
F08	Timeline Generation	D	• **GUI Video:** **EECS3311_GUI.mp4** ○ **Timestamp:** 0:12:01 - 0:12:22
F09	Multi-Source Comparison	AI	• **GUI Video:** **EECS3311_GUI.mp4** ○ **Timestamp:** 0:07:43 - 0:11:54 (timeout error at 0:11:09 ⇒ screen recording while calling on LLM consumes resources! This feature will work when the screen is not being

			recorded.)
F10	Claim Extraction	AI	• **GUI Video: EECS3311_GUI.mp4** ○ **Timestamp:** 0:05:07 - 0:06:07 (output at 0:06:00)
F11	Source Linking	D	• **GUI Video: EECS3311_GUI.mp4** ○ **Timestamp:** 0:06:31 - 0:07:11 • **CLI Video: EECS3311_CLI.mp4** ○ **Timestamp:** 0:01:31 - 0:01:39
F12	Topic Tracking	H	• **GUI Video: EECS3311_GUI.mp4** ○ **Timestamp:** 0:12:44 - 0:13:58
F13	Personalized Feed	H	• **GUI Video: EECS3311_GUI.mp4** ○ **Timestamp:** 0:13:59 - 0:14:26
F14	Saved Articles	D	• **GUI Video: EECS3311_GUI.mp4** ○ **Timestamp:** 0:06:08 - 0:06:30
F15	Natural Language Query	AI	• **GUI Video: EECS3311_GUI.mp4** ○ **Timestamp:** 0:14:59 - 0:15:52
F16	Daily Brief Generation	H	• **GUI Video: EECS3311_GUI.mp4** ○ **Timestamp:** 0:14:28 - 0:14:55
F17	Search History	D	• **GUI Video: EECS3311_GUI.mp4** ○ **Timestamp:** 0:12:24 - 0:12:43